R&D Report: Docker and CI/CD Deployment

Docker packages an app so it runs the same way on every machine, and CI/CD automatically tests and ships each change. Together they replace "it works on my machine" and hand-done deployments with a process you can repeat.

Why this matters

Most deployment trouble comes from two things. The app behaves differently on different machines, and a release depends on someone remembering every step. Docker deals with the first problem. CI/CD deals with the second.

Docker

Docker puts an app and everything it needs (the runtime, libraries and settings) into one package called a container. Whether it runs on a laptop or a cloud server, it behaves the same. A virtual machine does a similar job, but it carries a whole operating system. A container shares the host's system, so it is smaller and starts in seconds.

The four terms worth knowing:

- **Dockerfile:** the instructions for building the package.
- **Image:** the finished, read-only package built from those instructions.
- **Container:** an image that is currently running.
- **Registry:** where images are stored and shared, like Docker Hub.

Here is a small Dockerfile for a Node.js backend:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

The package files are copied before the rest of the code on purpose. Docker caches each step, so when only the code changes it skips the slow dependency install. When an app also needs a database, Docker Compose starts everything together with one command, `docker compose up`.

CI/CD

Continuous Integration (CI) means every time someone pushes code, it is built and tested automatically, so mistakes show up early. Continuous Delivery or Deployment (CD) means code that passes the tests is then released automatically, either after a manual approval or straight to production.

A simple pipeline looks like this:

1. A developer pushes code to GitHub.
2. The pipeline installs dependencies and runs the tests.
3. If the tests pass, it builds a Docker image.
4. The image is pushed to a registry.
5. The server pulls the new image and restarts the container.

GitHub Actions is the easiest tool to start with, because the code and the pipeline live in the same place. Jenkins and GitLab CI do the same job and are common in larger teams.

Using them together

Docker makes sure what gets deployed is identical to what was tested. CI/CD makes sure the testing and deploying happen the same way every time. A minimal GitHub Actions file looks like this:

```
on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t myapp:${{ github.sha }} .
```

The `needs: test` line means the image is only built if the tests pass. A real pipeline would add a login to the registry and a deploy step, with passwords kept in GitHub Secrets instead of the code.

Things to watch out for

- Never put passwords or keys inside an image or the repository.
- Use pinned versions like `node:20-alpine`, not `latest`, so builds stay predictable.
- Containers are temporary, so keep database data in a volume or a managed service.
- Slow or flaky tests slow down the whole team, so keep them quick and reliable.

Conclusion

Docker and CI/CD are worth learning early because they make releases boring in a good way. Docker removes the differences between machines, and CI/CD removes the manual steps. For a small full-stack project, Docker Compose plus GitHub Actions is enough, and tools like Kubernetes can wait until the project actually needs them.